

Transolver: A Fast Transformer Solver for PDEs

1 Reference

Transolver is a physics-attention-based Transformer architecture designed for solving partial differential equations (PDEs) on irregular geometries.

Z. Wu, M. Wang, and others, “Transolver: A Fast Transformer Solver for PDEs on General Geometries,” *International Conference on Machine Learning (ICML)*, 2024.

arXiv: [arXiv:2402.02366](https://arxiv.org/abs/2402.02366).

2 Introduction

Transolver is a neural operator architecture designed to learn mappings between physical fields defined on general geometries.

Given a parameterized PDE,

$$\mathcal{F}(u; \lambda) = 0, \quad x \in \Omega, \quad (1)$$

the objective is to learn the solution operator

$$\mathcal{G} : \lambda \mapsto u. \quad (2)$$

The central difficulty is that PDE solutions are functions defined over continuous spatial domains, while Transformer architectures normally operate on a finite sequence of tokens.

Transolver addresses this problem using **physics-inspired attention** and **slice-based representations**.

3 Discretized PDE Representation

Consider a physical domain

$$\Omega \subset \mathbb{R}^d. \quad (3)$$

Discretize the domain using N points,

$$X = \{x_i\}_{i=1}^N. \quad (4)$$

At every point, the physical state is represented by

$$u_i = u(x_i). \quad (5)$$

The complete discretized field is

$$U = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix}. \quad (6)$$

The neural network receives the coordinate and physical information,

$$h_i^0 = P_\theta(x_i, a_i), \quad (7)$$

where a_i denotes the input physical parameters.

Thus,

$$H^0 \in \mathbb{R}^{N \times d}. \quad (8)$$

4 Why Standard Self-Attention is Expensive

Standard self-attention computes

$$Q = HW_Q, \quad (9)$$

$$K = HW_K, \quad (10)$$

and

$$V = HW_V. \quad (11)$$

The attention matrix is

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right). \quad (12)$$

For N spatial points,

$$A \in \mathbb{R}^{N \times N}. \quad (13)$$

Therefore the memory and computational requirements grow quadratically with the number of points,

$$\mathcal{O}(N^2). \quad (14)$$

For high-resolution PDE simulations, this becomes prohibitively expensive.

5 Physics-Attention

Transolver replaces direct point-to-point attention with a physics-aware representation based on slices.

The fundamental idea is to divide the spatial points into a smaller number of latent physical slices.

Let

$$S \quad (15)$$

denote the number of slices, with

$$S \ll N. \quad (16)$$

Instead of computing interactions among all N points, Transolver first maps the points into these S slices.

6 Slice Projection

Let the input representation be

$$H = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_N \end{bmatrix} \in \mathbb{R}^{N \times d}. \quad (17)$$

A learnable projection generates slice scores,

$$P = HW_s, \quad (18)$$

where

$$W_s \in \mathbb{R}^{d \times S}. \quad (19)$$

Thus,

$$P \in \mathbb{R}^{N \times S}. \quad (20)$$

The score associated with point i and slice s is

$$p_{is} = h_i^T w_s. \quad (21)$$

A normalized slice assignment is obtained using softmax,

$$\alpha_{is} = \frac{\exp(p_{is})}{\sum_{r=1}^S \exp(p_{ir})}. \quad (22)$$

Therefore,

$$\sum_{s=1}^S \alpha_{is} = 1. \quad (23)$$

Each physical point is consequently represented as a weighted combination of the latent physical slices.

7 Slice Representation

The feature representation of slice s is obtained by aggregating the point features assigned to it.

A weighted aggregation is

$$z_s = \frac{\sum_{i=1}^N \alpha_{is} h_i}{\sum_{i=1}^N \alpha_{is}}. \quad (24)$$

Therefore the complete slice representation is

$$Z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_S \end{bmatrix} \in \mathbb{R}^{S \times d}. \quad (25)$$

The original N physical points have therefore been compressed into S physical tokens.

8 Physics-Attention

Self-attention is then performed over the slice representation.

Define

$$Q = ZW_Q, \quad (26)$$

$$K = ZW_K, \quad (27)$$

and

$$V = ZW_V. \quad (28)$$

The slice-level attention matrix is

$$A_s = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right). \quad (29)$$

Since

$$A_s \in \mathbb{R}^{S \times S}, \quad (30)$$

the computational cost is

$$\mathcal{O}(S^2), \quad (31)$$

rather than

$$\mathcal{O}(N^2). \quad (32)$$

The slice interaction is

$$\tilde{Z} = A_s V. \quad (33)$$

9 Projection Back to Physical Space

The slice representation must be mapped back to the original spatial points.

For point i ,

$$\tilde{h}_i = \sum_{s=1}^S \alpha_{is} \tilde{z}_s. \quad (34)$$

In matrix notation,

$$\boxed{\tilde{H} = \alpha \tilde{Z}}. \quad (35)$$

Here,

$$\alpha \in \mathbb{R}^{N \times S}. \quad (36)$$

Thus the complete slice-based attention mechanism is

$$H \xrightarrow{\text{slice projection}} Z \xrightarrow{\text{attention}} \tilde{Z} \xrightarrow{\text{back projection}} \tilde{H}. \quad (37)$$

10 Transolver Block

A Transolver block can be written schematically as

$$H' = H + \text{PhysicsAttention}(\text{LN}(H)). \quad (38)$$

A feed-forward network is subsequently applied,

$$H^{l+1} = H' + \text{MLP}(\text{LN}(H')). \quad (39)$$

Therefore,

$$\boxed{H^{l+1} = H' + \text{MLP}(\text{LN}(H'))}. \quad (40)$$

11 Multi-Head Physics-Attention

For M attention heads, each head has its own projections,

$$Q_m = ZW_Q^m, \quad (41)$$

$$K_m = ZW_K^m, \quad (42)$$

$$V_m = ZW_V^m. \quad (43)$$

Each head computes

$$O_m = \text{softmax}\left(\frac{Q_m K_m^T}{\sqrt{d_m}}\right) V_m. \quad (44)$$

The heads are concatenated,

$$O = \text{Concat}(O_1, \dots, O_M), \quad (45)$$

and projected,

$$O_{\text{out}} = OW_O. \quad (46)$$

12 Encoder

The input field is first lifted into a latent feature space,

$$H^0 = P_\theta(X, A). \quad (47)$$

A sequence of Transolver blocks is then applied,

$$H^{l+1} = \mathcal{T}_\theta^l(H^l). \quad (48)$$

After L layers,

$$H^L = \mathcal{T}_\theta^L \circ \dots \circ \mathcal{T}_\theta^1(H^0). \quad (49)$$

13 Output Projection

The final latent representation is projected onto the desired physical variables,

$$\hat{u}_i = Q_\theta(h_i^L). \quad (50)$$

Consequently, the learned operator is

$$\boxed{\mathcal{G}_\theta = Q_\theta \circ \mathcal{T}_\theta^L \circ \dots \circ \mathcal{T}_\theta^1 \circ P_\theta}. \quad (51)$$

14 Computational Complexity

Standard self-attention over N physical points requires

$$\mathcal{O}(N^2 d) \quad (52)$$

operations.

Transolver performs attention over S slices, where

$$S \ll N. \quad (53)$$

The slice attention therefore requires

$$\mathcal{O}(S^2 d). \quad (54)$$

The point-to-slice and slice-to-point operations require approximately

$$\mathcal{O}(N S d). \quad (55)$$

Hence the dominant attention-related complexity becomes

$$\boxed{\mathcal{O}(N S d + S^2 d)}. \quad (56)$$

Compared with

$$\mathcal{O}(N^2 d), \quad (57)$$

this can provide a substantial reduction when

$$S \ll N. \quad (58)$$

15 Training Objective

Given training pairs

$$\mathcal{D} = \left\{ (a^{(n)}, u^{(n)}) \right\}_{n=1}^{N_{\text{train}}}, \quad (59)$$

the prediction is

$$\hat{u}^{(n)} = \mathcal{G}_\theta \left(a^{(n)} \right). \quad (60)$$

The mean-squared-error loss is

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N_{\text{train}}} \sum_{n=1}^{N_{\text{train}}} \left\| \hat{u}^{(n)} - u^{(n)} \right\|_2^2. \quad (61)$$

The relative L^2 error is

$$\mathcal{E}_{\text{rel}} = \frac{\|\hat{u} - u\|_2}{\|u\|_2}. \quad (62)$$

16 PyTorch Implementation

```
1
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6
7 class PhysicsAttention(nn.Module):
8
9     def __init__(
10         self,
11         dim,
12         num_slices=32,
13         heads=4
14     ):
15         super().__init__()
16
17         assert dim % heads == 0
18
19         self.dim = dim
20         self.num_slices = num_slices
21         self.heads = heads
22         self.head_dim = dim // heads
23
24         # Point -> slice assignment
25         self.slice_projection = nn.Linear(
26             dim,
27             num_slices
28         )
29
30         # Slice attention
31         self.q = nn.Linear(
32             dim,
33             dim
34         )
```

```

35
36     self.k = nn.Linear(
37         dim,
38         dim
39     )
40
41     self.v = nn.Linear(
42         dim,
43         dim
44     )
45
46     self.out = nn.Linear(
47         dim,
48         dim
49     )
50
51     def forward(self, x):
52
53         B, N, C = x.shape
54
55         # -----
56         # Point -> slice assignment
57         # -----
58
59         logits = self.slice_projection(x)
60
61         assignment = F.softmax(
62             logits,
63             dim=-1
64         )
65
66         # -----
67         # Aggregate physical points into slices
68         # -----
69
70         weights = assignment.transpose(
71             1,
72             2
73         )
74
75         denominator = (
76             weights.sum(
77                 dim=-1,
78                 keepdim=True
79             )
80             + 1e-6
81         )
82
83         slices = torch.bmm(
84             weights,
85             x
86         )
87
88         slices = (
89             slices
90             / denominator
91         )
92

```



```

93 # -----
94 # Multi-head attention on slices
95 # -----
96
97 S = self.num_slices
98
99 q = self.q(slices)
100 k = self.k(slices)
101 v = self.v(slices)
102
103 q = q.view(
104     B,
105     S,
106     self.heads,
107     self.head_dim
108 ).transpose(1, 2)
109
110 k = k.view(
111     B,
112     S,
113     self.heads,
114     self.head_dim
115 ).transpose(1, 2)
116
117 v = v.view(
118     B,
119     S,
120     self.heads,
121     self.head_dim
122 ).transpose(1, 2)
123
124 scores = torch.matmul(
125     q,
126     k.transpose(-2, -1)
127 )
128
129 scores = (
130     scores
131     / (self.head_dim ** 0.5)
132 )
133
134 attention = F.softmax(
135     scores,
136     dim=-1
137 )
138
139 output = torch.matmul(
140     attention,
141     v
142 )
143
144 output = output.transpose(
145     1,
146     2
147 ).contiguous()
148
149 output = output.view(
150     B,

```

```

151         S,
152         C
153     )
154
155     output = self.out(
156         output
157     )
158
159     # -----
160     # Slice -> point reconstruction
161     # -----
162
163     reconstructed = torch.bmm(
164         assignment,
165         output
166     )
167
168     return reconstructed
169
170
171 class TransolverBlock(nn.Module):
172
173     def __init__(
174         self,
175         dim,
176         num_slices=32,
177         heads=4,
178         mlp_ratio=4
179     ):
180         super().__init__()
181
182         self.norm1 = nn.LayerNorm(
183             dim
184         )
185
186         self.attention = PhysicsAttention(
187             dim=dim,
188             num_slices=num_slices,
189             heads=heads
190         )
191
192         self.norm2 = nn.LayerNorm(
193             dim
194         )
195
196         hidden = dim * mlp_ratio
197
198         self.mlp = nn.Sequential(
199             nn.Linear(
200                 dim,
201                 hidden
202             ),
203             nn.GELU(),
204             nn.Linear(
205                 hidden,
206                 dim
207             )
208         )

```

```

209
210     def forward(self, x):
211
212         x = (
213             x
214             +
215             self.attention(
216                 self.norm1(x)
217             )
218         )
219
220         x = (
221             x
222             +
223             self.mlp(
224                 self.norm2(x)
225             )
226         )
227
228         return x
229
230
231 class Transolver(nn.Module):
232
233     def __init__(
234         self,
235         coord_dim,
236         in_channels,
237         out_channels,
238         width=128,
239         depth=4,
240         num_slices=32,
241         heads=4
242     ):
243         super().__init__()
244
245         self.input_projection = nn.Linear(
246             coord_dim + in_channels,
247             width
248         )
249
250         self.blocks = nn.ModuleList(
251             [
252                 TransolverBlock(
253                     dim=width,
254                     num_slices=num_slices,
255                     heads=heads
256                 )
257                 for _ in range(depth)
258             ]
259         )
260
261         self.output_projection = nn.Sequential(
262             nn.Linear(
263                 width,
264                 width
265             ),
266             nn.GELU(),

```

```

267         nn.Linear(
268             width,
269             out_channels
270         )
271     )
272
273     def forward(
274         self,
275         coords,
276         features
277     ):
278
279         x = torch.cat(
280             [
281                 coords,
282                 features
283             ],
284             dim=-1
285         )
286
287         x = self.input_projection(
288             x
289         )
290
291         for block in self.blocks:
292
293             x = block(x)
294
295         return self.output_projection(
296             x
297         )
298
299
300 if __name__ == "__main__":
301
302     device = (
303         "cuda"
304         if torch.cuda.is_available()
305         else "cpu"
306     )
307
308     batch_size = 2
309     n_points = 1024
310
311     coords = torch.rand(
312         batch_size,
313         n_points,
314         2,
315         device=device
316     )
317
318     features = torch.randn(
319         batch_size,
320         n_points,
321         1,
322         device=device
323     )
324

```

```

325     model = Transolver(
326         coord_dim=2,
327         in_channels=1,
328         out_channels=1,
329         width=128,
330         depth=4,
331         num_slices=32,
332         heads=4
333     ).to(device)
334
335     prediction = model(
336         coords,
337         features
338     )
339
340     print(
341         "Input:",
342         features.shape
343     )
344
345     print(
346         "Prediction:",
347         prediction.shape
348     )

```

17 Training

```

1
2 optimizer = torch.optim.AdamW(
3     model.parameters(),
4     lr=1e-3,
5     weight_decay=1e-4
6 )
7
8 criterion = nn.MSELoss()
9
10 for epoch in range(100):
11
12     model.train()
13
14     optimizer.zero_grad()
15
16     prediction = model(
17         train_coords,
18         train_input
19     )
20
21     loss = criterion(
22         prediction,
23         train_target
24     )
25
26     loss.backward()
27
28     optimizer.step()
29

```

```

30     if epoch % 10 == 0:
31
32         print(
33             f"Epoch {epoch}: "
34             f"Loss = {loss.item():.6e}"
35         )

```

18 Summary

The central mechanism of Transolver is the conversion of a large set of physical points into a smaller set of physics-aware slices.

The point-to-slice assignment is

$$\alpha_{is} = \text{softmax}_s(h_i^T w_s). \quad (63)$$

The slice representation is

$$z_s = \frac{\sum_{i=1}^N \alpha_{is} h_i}{\sum_{i=1}^N \alpha_{is}}. \quad (64)$$

Attention is then performed in the slice space,

$$\tilde{Z} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (65)$$

The representation is mapped back to the physical points,

$$\tilde{H} = \alpha \tilde{Z}. \quad (66)$$

Thus the principal architecture is

$$\boxed{\text{Physical Points} \rightarrow \text{Physics Slices} \rightarrow \text{Slice Attention} \rightarrow \text{Physical Points}}. \quad (67)$$

The key computational advantage is

$$\boxed{\mathcal{O}(N^2) \longrightarrow \mathcal{O}(NS + S^2), \quad S \ll N.} \quad (68)$$